

Informe: Implementacion Automatizada de una API REST de Gestion de Pedidos en AWS

Grupo de trabajo

June 8, 2026

1 Resumen

Se desarrollo una API REST para una plataforma basica de gestion de pedidos. La solucion permite crear pedidos, listar pedidos con filtros y ordenamiento, consultar pedidos por identificador y actualizar su estado. Ademas, se definio una automatizacion DevOps en AWS usando control de versiones, integracion continua, generacion de artefactos y despliegue automatico hacia un entorno de pruebas.

2 Requisitos de negocio

Requisito	Cumplimiento
Crear pedidos	Se implemento el endpoint <code>POST /orders</code> , que recibe cliente, productos, cantidades y direccion de envio. El pedido queda con estado inicial Pendiente .
Visualizar pedidos	Se implemento <code>GET /orders</code> . Permite filtrar por estado y ordenar por fecha de creacion, estado o cliente.
Actualizar estado	Se implemento <code>PATCH /orders/{id}/status</code> . Los estados permitidos son Pendiente , En Proceso , Enviado y Entregado .

3 Arquitectura de la aplicacion

La API fue desarrollada con Node.js usando el modulo HTTP nativo, sin dependencias externas obligatorias. Esta decision permite ejecutar pruebas y builds de forma simple en AWS CodeBuild. Para fines academicos, los pedidos se almacenan en memoria; en un escenario productivo se recomienda incorporar Amazon DynamoDB o Amazon RDS.

3.1 Endpoints principales

- GET /health: verifica disponibilidad del servicio.
- POST /orders: crea un pedido.
- GET /orders: lista pedidos con filtros y ordenamiento.
- GET /orders/{id}: consulta un pedido específico.
- PATCH /orders/{id}/status: actualiza el estado del pedido.

4 Automatizacion DevOps en AWS

4.1 Control de versiones

El código fuente y la infraestructura como código se gestionan en un repositorio AWS CodeCommit. La plantilla de CloudFormation crea el repositorio base para el proyecto.

Evidencia sugerida: insertar captura del repositorio CodeCommit con los archivos `src/`, `buildspec.yml`, `Dockerfile` e `infrastructure/cloudformation-pipeline.yml`.

4.2 Integración continua

AWS CodePipeline se configura para iniciar el flujo cuando se confirman cambios en la rama principal. AWS CodeBuild usa el archivo `buildspec.yml` para instalar dependencias, ejecutar pruebas unitarias y preparar el artefacto de despliegue.

Evidencia sugerida: insertar captura de CodeBuild mostrando una ejecución exitosa y el log con `npm test`.

4.3 Entrega continua

El pipeline incluye una etapa de despliegue hacia AWS Elastic Beanstalk. El artefacto generado por CodeBuild contiene el código de la API y el `Procfile`, por lo que Elastic Beanstalk puede iniciar el servidor con Node.js.

Evidencia sugerida: insertar captura de CodePipeline con las etapas Source, Build y DeployTest en verde.

4.4 Monitoreo

Elastic Beanstalk publica eventos y métricas en Amazon CloudWatch. Esto permite revisar logs, errores y estado del entorno desplegado.

Evidencia sugerida: insertar captura de eventos de Elastic Beanstalk o logs en CloudWatch.

5 Infraestructura como codigo

Se creo la plantilla `infrastructure/cloudformation-pipeline.yml`, que provisiona:

- Repositorio AWS CodeCommit.
- Bucket S3 para artefactos.
- Proyecto AWS CodeBuild.
- Pipeline AWS CodePipeline.
- Aplicacion y entorno AWS Elastic Beanstalk.
- Roles IAM necesarios para CodeBuild y CodePipeline.

6 Pruebas

Las pruebas unitarias verifican:

- Creacion de pedidos validos.
- Listado y consulta de pedidos.
- Actualizacion del estado.
- Rechazo de datos incompletos o estados invalidos.

7 Conclusiones

La solucion cumple los requisitos funcionales del caso de estudio y define un flujo DevOps automatizado en AWS. El pipeline permite que cada cambio confirmado en el repositorio sea construido, probado y desplegado automaticamente. Como mejora futura, se recomienda agregar persistencia con DynamoDB o RDS, autenticacion de usuarios y despliegue separado para ambientes de pruebas y produccion.